C++ 函数与面向对象

第二课 / 参数怎样传递，对象怎样协作

交换之后会发生什么?

```
int x = 10, y = 20;
swap1(x, y);
std::cout << x << ' ' << y << '\n';
```

先留下你的预测，稍后回来验证

函数

调用表达式怎样得到一个结果

test_add 的接口

```
int test_add(int a, int b) {
    int sum = a + b;
    return sum;
}

int result = test_add(1, 2);
```

参数 a、b 得到 1、2，result 得到 3

一次调用的局部状态

```
call                    a     b     sum
test_add(1, 2)          1     2       3
return 3              local objects end
caller                result = 3
```

返回的是结果值，局部变量不会变成全局变量

输出与返回值

```cpp
int test_add(int a, int b) {
    std::cout << "adding\n";
    return a + b;
}

int result = test_add(2, 3);
```

屏幕输出 adding，变量 result 得到 5

返回值可以继续参与表达式

```cpp
int total = test_add(2, 3) * 10;
std::cout << total << '\n';
```

先得到 5，再乘 10，最后输出 50

声明、定义与 void

```cpp
int test_add(int a, int b);

void hello() {
    std::cout << "Hello\n";
}
```

声明提供接口，定义提供函数体
void 表示没有返回值

同名局部变量互不干扰

```
void change(int value) {
    value = 99;
}

int value = 10;
change(value);
```

函数内外名字相同，也仍是不同对象

引用与指针

名字、地址和地址里的对象

引用是同一对象的别名

```
std::string band1("poppin party");
std::string& live1 = band1;
live1 = "afterglow";
```

band1 现在是什么?

引用赋值不会重新绑定

```
int a = 10, b = 20;
int& r = a;
r = b;
b = 30;
```

最后 a=20、b=30、r=20

指针保存地址

```
std::string* anime1 = &band1;

std::cout << *anime1 << '\n';
*anime1 = "roselia";
```

&band1 取地址，*anime1 访问目标对象

声明、取地址与解引用

```
int value = 10;
int* p = &value;

// int*    : type of p
// &value  : address of value
// *p      : object at that address
```

同一个符号，先看它出现在哪种语法里

改指针与改目标

```
std::string a("A"), b{"B"};
std::string* p = &a;
p = &b;
*p = "X";
```

a 是 A，b 是 X
p 的地址值变了，随后写入新的目标

空指针与有效期

```cpp
std::string* p = nullptr;
if (p != nullptr) {
    std::cout << *p << '\n';
}
```

先确认目标存在，再解引用

只读借用与生命周期

```
const std::string& view = band1;
const std::string* ptr = &band1;

// view = "X"; // not allowed
// *ptr = "X"; // not allowed
```

const 限制这条访问路径的修改能力
借用者使用时，原对象必须仍然有效

练习：追踪同一份数据

```
int a = 10, b = 20;
int& r = a;
int* p = &b;
r = 11;
*p = r;
p = &a;
```

最后 a、b、r、*p 分别是多少?

追踪题的逐行结果

operation	a	b	r	*p
r = 11	11	20	11	20
*p = r	11	11	11	11
p = &a	11	11	11	11

最后一行改变了指针的目标，没有给 a 或 b 赋值

交换实验

参数的类型决定函数能访问哪份数据

swap1 交换的是副本

```
void swap1(int a, int b) {
    int temp = a;
    a = b;
    b = temp;
}

swap1(x, y);
```

调用者 x=10、y=20

函数内部 a=20、b=10

swap1 的状态轨迹

step	x	y	a	b	temp
call	10	20	10	20	–
temp=a	10	20	10	20	10
a=b	10	20	20	20	10
b=temp	10	20	20	10	10

x、y 从头到尾没有被写入

swap2 通过地址写回

```
void swap2(int* a, int* b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

swap2(&x, &y);
```

参数复制的是地址，*a 和 *b 指向原对象

swap2 的状态轨迹

step	target of a	target of b
call	x=10	y=20
temp=*a	x=10	y=20
*a=*b	x=20	y=20
*b=temp	x=20	y=10

指针本身是副本，写入目标仍是调用者的对象

引用参数的交换

```
void swap_ref(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}

swap_ref(x, y);
```

a、b 直接成为 x、y 的别名

三种参数接口的选择

```
void f(int value);          // independent copy
void f(int& value);         // may modify caller
void f(const int& value); // read-only borrow
void f(int* value);         // pointer value
```

接口应说明是否修改、是否允许没有对象

练习：修复不能写回的函数

```
void add_bonus(int score) {
    score += 5;
}
```

要求调用 add_bonus(score) 后，原分数增加 5
你只需要改哪里?

修改原对象与返回新值

```
void add_bonus(int& score) {
    score += 5;
}

int with_bonus(int score) {
    return score + 5;
}
```

add_bonus(score) 直接修改

score = with_bonus(score) 显式接收新值

课间休息

10 分钟

回来后扩展 Band，再一起追踪完整程序

Band：数据与行为

同一支乐队有自己的成员和操作

从记录到 Band

```
class Band {
private:
    std::string keyboard;
public:
    void set_keyboard_anon() {
        keyboard = "anon";
    }
};
```

外部通过公开方法改变内部状态

五个字段属于同一支乐队

```
guitar    mutsumi
vocal     hatsune
drum      nyamu
bass      umiri
keyboard  saki
```

字段名属于类型，每个对象保存自己的一组值

对象各自保存状态

```
Band a;
Band b;
a.set_keyboard_anon();
```

a 的变化会自动影响 b 吗?

两个对象的独立状态

```
before          a.keyboard=""   b.keyboard=""
a.set_keyboard_anon()
after           a.keyboard="anon"
                b.keyboard=""
```

相同成员名，不代表同一份存储

private 与 public

```
a.keyboard = "anon";  // error
a.set_keyboard_anon(); // OK
```

成员方法可以访问自己的 private 成员

封装提供统一的修改入口

```
void set_keyboard(const std::string& name) {
    if (name.empty()) return;
    keyboard = name;
}
```

调用者提出修改，类决定是否接受

构造函数给对象初始状态

```
Band(std::string key) {
    keyboard = key;
}

Band a("saki");
```

名称与类相同，不写返回类型

有参构造与默认构造

```
Band a("saki"); // OK in the reduced class
// Band b;      // no matching constructor
```

声明了这个有参构造后，编译器不会再隐式生成默认构造

成员初始化列表

```
Band(std::string key)
    : keyboard(key) {}
```

keyboard 直接用 key 初始化
成员按声明顺序初始化

声明顺序决定初始化顺序

```
class Example {
    int first;
    int second;
public:
    Example() : second(2), first(1) {}
};
```

实际仍先 first，再 second
初始化列表按声明顺序写更清楚

读方法与写方法

```
void print_info() const {
    std::cout << keyboard << '\n';
}

void set_keyboard_anon() {
    keyboard = "anon";
}
```

成员函数末尾的 const 承诺不修改普通成员

const 对象可以调用什么?

```
const Band band("saki");
band.print_info();          // OK
// band.set_keyboard_anon(); // error
```

只读承诺也成为调用接口的一部分

练习：独立完成一个 Band

保留一个 private 字符串 keyboard

用构造函数接收初始值

提供只读打印和修改方法

创建两个对象，验证只改动其中一个

Band 练习的验收

a 初值 saki，b 初值 eve

只修改 a 后，输出应为 anon / eve

const 对象可以打印

类外直接写 keyboard 应编译失败

完整 Band 的五个成员

```
Band ave_mujica(
    "mutsumi", "hatsune", "nyamu",
    "umiri", "saki"
);
ave_mujica.print_info();
ave_mujica.live();
```

guitar、vocal、drum、bass、keyboard

vector 存放多支乐队

```
std::vector<Band> live_bands;
live_bands.emplace_back(
    "hina", "aya", "maya", "chisato", "eve");
live_bands.emplace_back(
    "sayo", "yukina", "ako", "lisa", "rinko");
```

vector 管理数量，Band 管理每个对象

vector 的大小与位置

```
std::cout << live_bands.size() << '\n';
live_bands[0].print_info();
live_bands.at(1).print_info();
```

两次追加后 size 为 2，有效下标为 0 和 1

遍历时要改哪一份?

```
for (Band band : live_bands) {
    band.set_keyboard_anon();
}

for (Band& band : live_bands) {
    band.set_keyboard_anon();
}
```

上面改副本，下面改容器里的对象

只读遍历完整乐队

```
for (const Band& band : live_bands) {
    band.print_info();
}
```

借用容器中的对象，并通过 const 接口读取

类与容器检查

值传递、引用、指针分别传来了什么?

构造函数在哪一步建立初始状态?

读方法为什么可以带 const?

继承

在 Band 的基础上增加 vocal_actor

MofoBand 的继承关系

```
class MofoBand : public Band {
protected:
    std::string vocal_actor;
public:
    void print_mofo_info() const {
        print_info();
        std::cout << vocal_actor << '\n';
    }
};
```

已有 Band 行为，再增加配音演员信息

基类先初始化

```
MofoBand(std::string gt, std::string vo,
         std::string dr, std::string ba,
         std::string key, std::string actor)
    : Band(gt, vo, dr, ba, key),
      vocal_actor(actor) {}
```

先 Band，再 vocal_actor，最后进入函数体

protected 的访问范围

MofoBand 的成员函数：可以

MofoBand_New 的成员函数：可以按继承规则访问

main 中直接写对象.vocal_actor：不可以

this 指向当前对象

```
void change_actor(
    const std::string& new_actor) {
    this->vocal_actor = new_actor;
}
```

左侧是当前对象的成员，右侧是参数

练习：演员信息的修改

创建一个 actor 初值为 amane 的对象

调用 change_actor("ayane")

输出修改后的信息

说明为什么 main 不能直接改 protected 字段

class_band 的运行轨迹

ave_mujica：创建、print_info、live

live_bands：加入两支乐队、遍历

ave_mujica：get_guitar、修改 keyboard

morfonica 与 lovelive：创建、修改 actor

主课回看

谁拥有这份数据?

函数访问的是副本还是原对象?

外部代码通过哪个接口修改它?

可选拓展

虚函数、切片与资源所有权

通过基类调用派生行为

```
struct Base {
    virtual void live() const { /* ... */ }
    virtual ~Base() = default;
};
struct Derived : Base {
    void live() const override { /* ... */ }
};
```

Base& 指向 Derived 时，live 调用派生实现

值复制可能发生切片

```
Derived d;
Base copied = d;
Base& borrowed = d;
```

copied 只保留 Base 部分
borrowed 仍引用完整的 d

独占资源的所有权

```
std::unique_ptr<Band> owner = std::make_unique<Band>(
    "mutsumi", "hatsune", "nyamu",
    "umiri", "saki");
std::unique_ptr<Band> next = std::move(owner);
```

所有权交给 next，owner 变空
离开作用域时自动释放

课后材料与挑战

必做：用本课接口完成 Assessment-02

加练：从例题索引选择薄弱知识点

挑战：读完拓展后再完成相关题目